# Mathematical Model of Compare-And-Swap (CAS)

This document details the mathematical formalization of the Compare-And-Swap (CAS) mechanism, demonstrating how it guarantees state consistency in a concurrent environment through linearizability, followed by a concrete numeric example and sequence diagram.

## 1. State Space and the Atomic Transition Function

Let $V$ be the set of all possible values that a shared memory register $R$ (e.g., a Redis key holding a token count) can hold. At any discrete logical time step $t$, the state of the register is denoted by $s_t \in V$.

A CAS operation is modeled as an atomic state transition function mapping the current state of the register, an expected value, and an update value to a new state and a boolean success flag.

$$f_{CAS}: V \times V \times V \to V \times \{\top, \bot\}$$

For a given state $s_t$, an expected value $e \in V$, and an update value $u \in V$, the transition is defined as:

$$f_{CAS}(s_t, e, u) = (u, \top) \text{ if } s_t = e$$

$$f_{CAS}(s_t, e, u) = (s_t, \bot) \text{ if } s_t \neq e$$

In the concurrency model, this transition is **indivisible**. No other operation can observe or modify the state of $R$ between evaluating $s_t = e$ and assigning $s_{t+1}$.

## 2. Numeric Example: Token Bucket Rate Limiting

Let us model a scenario where two concurrent threads (Thread A and Thread B) are processing requests for the same tenant. They both need to consume 1 token from the bucket. The current state (tokens available) is 5.

- **Initial State:** $s_0 = 5$
- Both threads read the state concurrently: $e_A = 5$, $e_B = 5$

- Both threads calculate their desired update value: $u_A = 4$, $u_B = 4$

Because the system is asynchronous, one thread's CAS operation will inevitably reach the shared register first. Let us trace the execution.

## 3. Concurrency Sequence Diagram

The following diagram illustrates the linearizability of the CAS operations, forcing a total order on the concurrent updates.

| Thread A | Shared State (Redis) | Thread B |
|---|---|---|
| Read state → 5 | **Tokens: 5** | Read state → 5 |
| Calculate new: 5 - 1 = 4 | | Calculate new: 5 - 1 = 4 |
| **Invoke:** $f_{CAS}(5, 5, 4)$ | | |
| | Evaluates: 5 == 5.<br>State transitions to 4. | |
| **Returns: (4, ⊤) [Success]** | **Tokens: 4** | **Invoke:** $f_{CAS}(4, 5, 4)$ |
| *Continues processing...* | Evaluates: 4 ≠ 5.<br>State remains 4. | |
| | | **Returns: (4, ⊥) [Fail]** |
| | | **Retry:** Read state → 4 |
| | | Calculate new: 4 - 1 = 3 |
| | | **Invoke:** $f_{CAS}(4, 4, 3)$ |
| | Evaluates: 4 == 4.<br>State transitions to 3. | |
| | **Tokens: 3** | **Returns: (3, ⊤) [Success]** |

**Consistency Guarantee:** If we did not use CAS (e.g., just a simple Read-Modify-Write without atomicity), both threads would read 5, both would calculate 4, and both would write 4. Two tokens would be logically consumed, but the bucket would show 4 tokens remaining (a loss of 1 token due to

> overwriting). CAS ensures that Thread B's initial write fails because the state shifted out from under it, forcing it to re-evaluate based on the new, correct reality.